

Fine-Grained Dynamic Update Controller Synthesis

ANONYMOUS AUTHOR(S)

Updating a running controlled system creates a mixed-version interval beyond the old and new controllers' separate safety guarantees. Components may become ready for replacement at different times, with requirements active across different intervals. Fine-Grained Dynamic Update Controller Synthesis (FG-DUCS) combines set-valued component transfer, requirement lifetimes and residuals, precedence, and compatible handover targets. Update commands and handover require uncontrollable quiescence.

We encode the problem as a finite reachability game and specialize its attractor computation with lazy successor generation. The terminating, sound and complete procedure certifies all entries/outcomes of the supplied finite model; execution guarantees are conditional on residual soundness (RS) and runtime assumptions A1–A4. It returns a policy/rank/handover certificate or a losing region. For the benchmark inputs, residual soundness is mechanically established for all 138 new-requirement initializations and 89 of 135 update-time initializations under the stated interpretations; the remaining 46 reference action predicates are unverified. Witnesses separate fine-grained from bulk updates and $K + 1$ discovered states from 2^K under full expansion. Raw-LTS expectations agree with 92 correctness and 14 capability jobs. On 27 instances from nine DUCS benchmarks, with a 64 GiB heap and 20-minute budget, FG-DUCS-OTF decides 26: 25 realizable and one unrealizable with a checked losing certificate. Explicit same-semantics solving decides 14 and takes 1.6–2,124× longer on their paired five-run medians; the fork's legacy synthesis path decides nine. Five legacy cells are unmeasured due to an author-side capture cap. On 90 hub games, paired median speedups are 5.3–118×; Travel models expose a shared resource limit at six or more concurrent agencies. Resource-limited cells remain unresolved.

CCS Concepts: • **Software and its engineering** → **Software evolution**.

Additional Key Words and Phrases: dynamic software updating, discrete-event controller synthesis, supervisory control, on-the-fly synthesis

ACM Reference Format:

Anonymous Author(s). 2027. Fine-Grained Dynamic Update Controller Synthesis. In *Proceedings of The ACM International Conference on the Foundations of Software Engineering (FSE 2027)*. ACM, New York, NY, USA, 21 pages.

1 Introduction

A deployed controlled system must evolve while its environment continues to act. A production cell may still hold a workpiece when its controller or equipment model changes; a workflow may have obligations left by an earlier request. Stopping operation can postpone the update, but separate verification of the old and new configurations does not explain how to move safely between them. Runtime evolution therefore needs a controller for the transition itself, connecting the assurance of stable configurations to the behavior permitted during change [3, 9, 19, 40].

Reconfiguration connects valid configurations through safe intermediate states [13, 25]; control may change with assumptions and goals [15]. State-transformer synthesis [43], combined-version verification [20] and distributed version consistency [5] address complementary concerns. We synthesize control between supplied transfers.

Components can become replaceable at different times, for example after releasing a resource. Requirements also have different lifetimes: old protection may remain necessary until replacement, while a new requirement inherits an obligation from the current state. Dynamic scenario updates expose such intermediate obligations [18, 30]. Synthesis must coordinate changes, ordinary operation and requirement boundaries throughout this interval.

Dynamic Update Controller Synthesis (DUCS) synthesizes control preserving safety and completing an update [33, 34]. Its native interface offers global environment reconfiguration and whole-set stop/start. Transfers may be set-valued; intermediate steps can be encoded in the new environment, and transition requirements permit specification overlap [34, Sec. 4.2]. Individual component/requirement boundaries need additional encoding. The GR(1) bridge similarly uses a switching condition [2]. We make these finer decisions explicit inputs and synthesize their order and outcome-dependent continuation.

FG-DUCS takes verified old/new closed loops, component transfer relations, individual requirement boundaries and residuals, precedence, and handover targets. Commands and handover require uncontrollable quiescence. In the Production Cell below, replacing the empty station, moving the product, and replacing the other station succeeds although no reachable state permits both replacements together.

Three difficulties remain: transfer outcomes can require different continuations; independent changes induce exponentially many configurations; and success must cover every reachable old-state entry and permitted outcome. FG-DUCS encodes a finite reachability game and explores successors lazily, returning a policy with decreasing ranks and compatible handover targets, or a losing region.

Strong planning and on-the-fly games provide the algorithmic foundation [10, 11, 42]. We contribute the update contract and encoding, lazy buckets, certificates and evaluation. The preliminary O-DUCS report explores updates from old reachable states [22]; FG-DUCS develops component/requirement boundaries and a terminating, sound and complete certifying procedure.

Contributions.

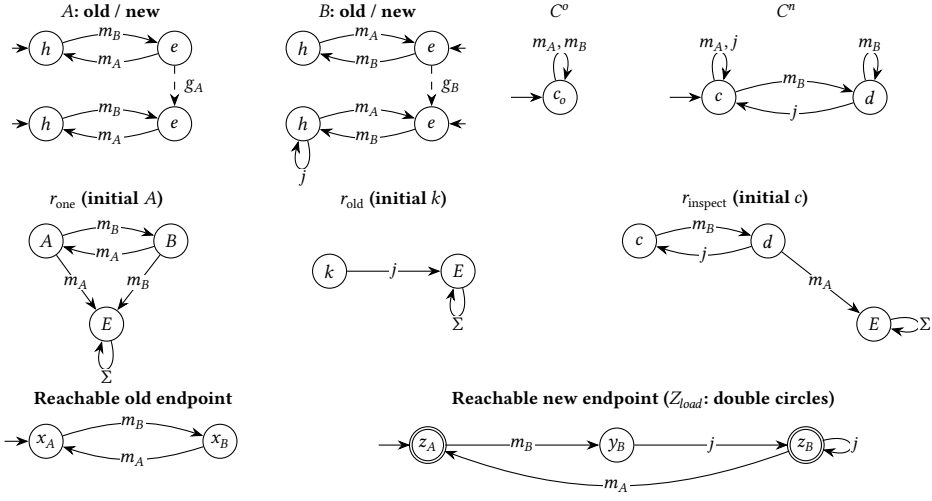
- **C1: Fine-grained formulation.** A finite contract combines set-valued component transfers, requirement lifetimes and residuals, precedence, and handover targets. Complete witnesses separate three granularity restrictions (Section 3.3).
- **C2: Certifying on-the-fly synthesis.** A terminating, sound, complete solver covers all roots/outcomes and returns a WIN certificate (σ, rk, κ) or LOSS certificate L (Theorem 4.3).
- **C3: Search separation.** An independent-update family requires $K + 1$ discovered states for lazy exploration and 2^K for Direct-Full, isolating successor-generation cost.
- **C4: Evaluation.** Raw-LTS checks validate decisions. On 27 Xeon instances, lazy synthesis decides 26 versus Direct-Full's 14 and reduces paired median time by 1.6–2,124×. Hub tests yield paired speedups of 5.3–118×; Travel exposes shared resource limits.

Guarantees are relative to the supplied finite model and RS; execution additionally assumes A1–A4. Benchmark residual guarantees use the scoped requirement meanings defined in Section 3.1 and Supplement S3.

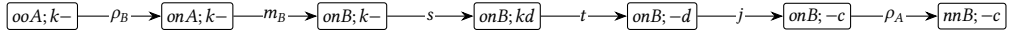
2 Motivating Example

Consider a finite Production Cell witness with two stations, A and B , sharing exactly one product [28, 34]. The product must be neither lost nor duplicated, and a station holding it cannot be replaced. Write h/e for holding/empty and o/n for old/new. The old controller moves the product between (A_h^o, B_e^o) and (A_e^o, B_h^o) . An update may be requested at either state, so one synthesized policy must handle both entries.

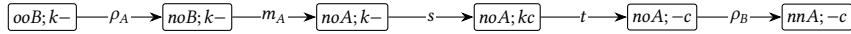
A bulk transfer applies both local transfer relations together. Each relation accepts only an empty station, so their product requires (A_e^o, B_e^o) . This state is unreachable while preserving one product. Waiting for a different old state cannot make bulk replacement possible. With separate transfer



Policy p_A (remaining rank 6, ..., 0)



Policy p_B (remaining rank 5, ..., 0)



$m_A/m_B/j = \text{move}_A/\text{move}_B/\text{inspect}$; $o/n, h/e, k/c/d/E/- = \text{old/new, holding/empty, ok/clear/pending/error/inactive}$. $\Sigma_N = \{m_A, m_B, j\}$, $\Sigma_{\text{upd}} = \{\rho_A, \rho_B, s, t\}$, $\Sigma_{\text{uc}} = \emptyset$; $s = \text{start}(r_{\text{inspect}}) < t = \text{stop}(r_{\text{old}})$. For $\forall_A \forall_B x; \mu_0 \mu_1$, $P = \{\rho_i : v_i = o\} \cup \{s : \mu_i = -\} \cup \{t : \mu_0 \neq -\}$; x is holder and r_{one} state. Initial plant states are (A_h^o, B_e^o) . Alphabets are the displayed outgoing labels over all local states; $g_i = \{(e_i^o, e_i^n)\}$. $R^{\text{old/new/upd}} = \{r_{\text{old}}\}/\{r_{\text{inspect}}\}/\{r_{\text{one}}\}$. Start_{one} is the two old one-product states; $t_{\text{one}} = \text{holder}$; its observation preserves moves (exact RS). Start_{inspect} contains all eight tagged one-product states; $t_{\text{inspect}} = d$ at B_h , otherwise c . Before activation, its observation erases every label except s . Thus c is exact at B_e ; at B_h , $\mathcal{L}(d) \subseteq \mathcal{L}(c)$ (witness m_A), a conservative residual. $x_A = (A_h^o, B_e^o, c, k)$, $x_B = (A_h^o, B_h^o, c, k)$; $z_A = (A_h^n, B_e^n, c, c)$, $y_B = (A_h^n, B_h^n, d, d)$, $z_B = (A_h^n, B_h^n, c, c)$. Q_0 consists of the first states of p_A, p_B ; their last states are goals with $\kappa = z_B, z_A$, respectively; $Z_{\text{load}} = \{z_A, z_B\}$. The displayed paths define σ (empty elsewhere), with 13 states and 11 edges. A bulk $g_A \times g_B$ requires unreachable (A_e^o, B_e^o) . Unlisted plant/controller transitions are empty; unlisted non-error monitor transitions self-loop; E absorbs every label. Comma-separated arrow labels denote separate edges. The four Post rules determine every bucket; the unsafe-inclusive closure census is in Supplement S1.

Fig. 1. Complete Production Cell witness $\mathcal{W}_{\text{cell}}$: local replacements, active requirements, endpoints and a policy from both entries. Dashed arrows are transfer relations, not ordinary events.

events, the entry with A holding instead admits

$$(A_h^o, B_e^o) \xrightarrow{\rho_B} (A_h^o, B_e^n) \xrightarrow{\text{move}_B} (A_e^o, B_h^n) \xrightarrow{\rho_A} (A_e^n, B_h^n).$$

The initially empty B is replaced first; moving the product then makes A replaceable. The symmetric entry reverses the component order. This component projection is only part of the update: the requirements determine where the remaining boundary and inspection events belong.

The interval monitor r_{one} tracks the holder throughout. Old r_{old} forbids inspection until its own stop, while new r_{inspect} requires inspection before the product next leaves B . After the first transfer and product move, A is old, B is new and holding, and r_{old} is still active. Starting r_{inspect} at this point installs its pending-obligation state d , rather than the initial clear state c . This is the role of residual initialization ι : activation must use a monitor state appropriate to the supplied history meaning. The precedence $\text{start}(r_{\text{inspect}}) < \text{stop}(r_{\text{old}})$ creates a brief overlap; stopping the old prohibition then

permits inspection and discharges the pending obligation. Figure 1 specifies the complete fixture and both policy paths; Section 3.3 proves the separation.

The returned policy tells an update orchestrator which controllable event to permit at each observed state. Its rank certifies finite progress for every modeled outcome, and its handover table identifies the compatible new-controller state to load. A LOSS region identifies an obstruction to the supplied contract, providing a concrete basis for revising that contract.

We connect this small witness to nine DUCS-derived benchmark models in Section 6. In Industry/R1, a checked losing region exposes the need for a validation step forbidden during the update. Permitting that event in a separate contract variant makes the update realizable; Section 6 preserves both the original LOSS and the variant result.

3 FG-DUCS

Let $v \in \{\text{old}, \text{new}\}$ denote the two versions. Each supplied closed loop $E^v \parallel C^v$ is separately verified against R^v ; CL^v instruments it with monitors. FG-DUCS controls the intervening component and requirement changes. Execution interpretation uses A1–A4 in Section 5.

3.1 System, Requirement, and Update Inputs

Finite LTSs and safety monitors. An LTS is $\mathcal{L} = (S, \Sigma_{\mathcal{L}}, \delta, s^0)$, where $\delta : S \times \Sigma_{\mathcal{L}} \rightarrow 2^S$ permits non-deterministic outcomes. Partition the global event set as $\Sigma = \Sigma_{\text{uc}} \uplus \Sigma_{\text{c}}$, where events in Σ_{uc} cannot be disabled. In a synchronous product, an event shared by components synchronizes only when every component having that event can take it; nonparticipating components retain their states [29]. Only components generate events; controllers and monitors neither generate them nor select outcomes. A safety requirement r has a distinct total deterministic monitor $t_r = (M_r, \Sigma, \eta_r, m_r^0, F_r)$, whose absorbing errors recognize violating prefixes [1, 27]. Write $\mathbb{T}_R = \{t_r : r \in R\}$ and F_t for the error states of monitor t ; an empty monitor product is a one-state error-free LTS. Monitors observe labels, whereas policies also observe successor states. Outcome-sensitive safety therefore requires distinguishing labels.

Definition 3.1 (Typed FG-DUCS update contract). *Component-level state transfer.* For each $i \in I$, let $e_i^{\text{old}}, e_i^{\text{new}}$ be the pre-update and post-update components and let $g_i \subseteq S_{e_i^{\text{old}}} \times S_{e_i^{\text{new}}}$ be their state-transfer relation. Define $g_i(s) = \{s' \mid (s, s') \in g_i\}$. The supplied g_i includes every modeled outcome of transformation, activation and rewiring. Let $E^v = \parallel_{i \in I} e_i^v$. Each component state carries its old/new version tag.

Requirement active periods and monitor initialization. Let pairwise-disjoint $R^{\text{old}}, R^{\text{new}}, R^{\text{upd}}$ contain the pre-update, post-update, and update-interval safety requirements, respectively. Old monitors inherit their monitored endpoint states at admission. For each $r \in R^{\text{new}} \cup R^{\text{upd}}$, rather than resetting away history, the input supplies a set Start_r of version-tagged component tuples and a *monitor-initialization map* $\iota_r : \text{Start}_r \rightarrow M_r \setminus F_r$. Residual-sound initialization requires

$$\mathcal{L}_r(\iota_r(\xi)) \subseteq \mathcal{C}_r(\xi) \quad (\xi \in \text{Start}_r). \quad (\text{RS})$$

Here $\mathcal{L}_r$ is the initialized monitor's safe continuation language and $\mathcal{C}_r$ the continuations compatible with every supplied activation history at ξ . RS concerns the supplied requirement/observation meanings; Supplement S3.1 distinguishes history-inclusive H, activation-safe A and entry-scoped E meanings and their exactness conditions. Unverified cases retain RS as an external premise. For $r \in R^{\text{upd}}$, Start_r includes every reachable old component state.

Verified pre-update and post-update closed loops and admissible handover targets. For $v \in \{\text{old}, \text{new}\}$, finite C^v preserves uncontrollables and makes $E^v \parallel C^v$ deadlock-free and satisfy every requirement in R^v . After hiding controller auxiliary events, its observation alphabet is $\bigcup_i \Sigma_{e_i^v}$. Their union over

both versions is the ordinary alphabet Σ_N . The monitored endpoints are

$$CL^{\text{old}} := E^{\text{old}} \parallel C^{\text{old}} \parallel \prod_{r \in R^{\text{old}}} t_r, \quad CL^{\text{new}} := E^{\text{new}} \parallel C^{\text{new}} \parallel \prod_{r \in R^{\text{new}}} t_r$$

Write Reach for initial-state reachability. Supply nonempty $Z_{\text{load}} \subseteq \text{Reach}(CL^{\text{new}})$ as the handover set, including component, new-controller and new-monitor states. Target reachability is checked. *One-shot update events and dependencies.* The set of update events, each executed exactly once, is

$$\Sigma_{\text{upd}} = \{\rho_i \mid i \in I\} \cup \{\text{stop}(r) \mid r \in R^{\text{old}}\} \cup \{\text{start}(r) \mid r \in R^{\text{new}}\}. \quad (1)$$

The events $\rho_i, \text{stop}(r), \text{start}(r)$ respectively update a component, deactivate a pre-update requirement monitor, and activate a post-update requirement monitor. Update events are distinct controllable events disjoint from ordinary events Σ_N ; that is, $\Sigma = \Sigma_N \uplus \Sigma_{\text{upd}}$ and $\Sigma_{\text{upd}} \subseteq \Sigma_c$. Even after choosing ρ_i , the controller cannot choose the outcome in g_i ; it is one model transition. Update events use the non-preemptive successor rule in Definition 3.2: they are enabled only when no uncontrollable ordinary event is enabled.

Let the strict partial order $< \subseteq \Sigma_{\text{upd}} \times \Sigma_{\text{upd}}$ express precedence among update operations. For a pending-event set P , event a is order-enabled exactly when $a \in P$ and every $a' < a$ has left P . Monitors self-loop on update events that they do not specify.

Eligibility. Eligibility checks the finite input's types, event ownership, controllability and monitor structure; safe, deadlock-free endpoint controllers that preserve uncontrollables; complete update-monitor entry domains; strict partial-order dependencies; and reachable load targets. RS is external evidence for the supplied initialization meanings. Supplement S2.1 gives every check; synthesis checks initialization types, domains and non-error values, while activation-history completeness remains an input obligation.

Put an interval-wide property in R^{upd} and one required after handover in R^{new} ; to overlap new and old protection, require $\text{start}(r^{\text{new}}) < \text{stop}(r^{\text{old}})$.

3.2 Update States and Transitions

Definition 3.2 (Update game induced by the contract). States and all entry states. An update state is a triple $q = (\xi, \mu, P)$. The set $P \subseteq \Sigma_{\text{upd}}$ contains the pending events, and

$$\xi(i) \in \{\text{old}\} \times S_{e_i^{\text{old}}} \uplus \{\text{new}\} \times S_{e_i^{\text{new}}}, \quad \rho_i \in P \iff \xi(i) \in \{\text{old}\} \times S_{e_i^{\text{old}}}. \quad (2)$$

μ stores states for $\mathbb{T} = \mathbb{T}_{R^{\text{old}}} \cup \mathbb{T}_{R^{\text{new}}} \cup \mathbb{T}_{R^{\text{upd}}}$, with $\mu(t_r) \in M_r \cup \{\perp\}$ for each $t_r \in \mathbb{T}$. $\perp$ denotes inactivity and belongs to no M_r . The set of active monitors is

$$\text{ActiveM}(P) = \mathbb{T}_{R^{\text{upd}}} \cup \{t_r \mid r \in R^{\text{old}}, \text{stop}(r) \in P\} \cup \{t_r \mid r \in R^{\text{new}}, \text{start}(r) \notin P\}, \quad (3)$$

and we require $t \in \text{ActiveM}(P) \iff \mu(t) \neq \perp$.

The function $\text{Init}(z)$ inherits component state ξ and pre-update monitor states from monitored pre-update closed-loop state z , leaves the post-update monitors inactive, initializes the update-interval monitors using $t_r(\xi)$, and sets $P = \Sigma_{\text{upd}}$. It does not inherit the C^{old} component of z , because update admission switches from the pre-update controller to the update orchestrator. The update-entry set is

$$Q_0 = \{\text{Init}(z) \mid z \in \text{Reach}(CL^{\text{old}})\}. \quad (4)$$

One policy covers all reachable old states; restricted admission is a different problem. For the cell, q_A^0/q_B^0 denote entry with A/B holding the product, so $Q_0 = \{q_A^0, q_B^0\}$.

Four kinds of implicit transitions. For $q = (\xi, \mu, P)$ with $\xi(i) = (v, s)$, call e_i^v the active local LTS of component i . Let $U(q)$ contain the uncontrollable ordinary events enabled by the active-component

synchronous product (including events whose outcomes are unsafe). The ordinary-event rule below determines $U(q)$ independently of update events and goal recognition. For every $a \in \Sigma_{\text{upd}}$, impose

$$U(q) \neq \emptyset \implies \text{Post}(q, a) = \emptyset. \quad (\text{NP})$$

When $U(q) = \emptyset$, the orchestrator may issue one enabled update command; the command occurs as one event with its outcome chosen from the full successor set. Define $\text{Post}(q, a)$ to contain exactly the successors below, preserving every unstated component of ξ, μ, P ; “advance T ” sets $\mu'(t) = \eta_t(\mu(t), a)$ for each $t \in T$.

Ordinary event $a \in \Sigma_N$. If a belongs to the local event set of at least one active component and every active component having a can transition, take every synchronous-product outcome and advance $\text{ActiveM}(P)$.

Component update ρ_i . If $U(q) = \emptyset$, order-enabled, and $\xi(i) = (\text{old}, s)$, set $\xi'(i) = (\text{new}, s')$ for each $s' \in g_i(s)$, delete ρ_i from P , and advance $\text{ActiveM}(P)$.

Requirement deactivation $\text{stop}(r)$. If $U(q) = \emptyset$ and order-enabled, advance $\text{ActiveM}(P) \setminus \{t_r\}$, retain inactive monitors at $\perp$, set $\mu'(t_r) = \perp$, and delete $\text{stop}(r)$ from P ; t_r does not observe it.

Requirement activation $\text{start}(r)$. If $U(q) = \emptyset$, order-enabled, and $\xi \in \text{Start}_r$, advance $\text{ActiveM}(P)$, retain other inactive monitors at $\perp$, install $\mu'(t_r) = \iota_r(\xi)$, and delete $\text{start}(r)$ from P ; t_r begins online observation afterward.

If a precondition is false, the successor set is empty; every nondeterministic outcome is included and environment-chosen. Let $\mathcal{Q}$ be the least set containing Q_0 and closed under all Post transitions, and let $\text{En}_{\text{uc}}(q), \text{En}_c(q)$ be the sets of uncontrollable and controllable events, respectively, whose successor sets are nonempty.

Safety and handover-ready goals. An update state $q = (\xi, \mu, P)$ is safe when no active monitor is in an error state:

$$\text{Safe}(q) \iff \forall t \in \text{ActiveM}(P) : \mu(t) \notin F_t. \quad (5)$$

For an update state q with $P = \emptyset$ and a state $z \in Z_{\text{load}}$, write $q \sim z$ when, component by component and monitor by monitor, the component states after removing the version tags and all post-update safety-monitor states agree between q and z . The handover map κ supplies the new-controller state from its selected z . Define the goal-state set as

$$Q_{\text{goal}} = \{q = (\xi, \mu, \emptyset) \in \mathcal{Q} \mid \text{Safe}(q) \wedge U(q) = \emptyset \wedge \exists z \in Z_{\text{load}} : q \sim z\}. \quad (6)$$

At a goal, the orchestrator disables ordinary controllables and loads $\kappa(q)$ through load, outside Σ and the update rank. Quiescence excludes UC competition; a non-quiescent match remains a non-goal and may be invalidated before loading.

State-dependent policies and finite completion. A memoryless update policy is a function

$$\sigma : \mathcal{Q} \setminus Q_{\text{goal}} \longrightarrow 2^{\Sigma_c}, \quad \sigma(q) \subseteq \text{En}_c(q). \quad (7)$$

The possible next states under this policy are

$$\text{Succ}_{\sigma}(q) = \bigcup_{u \in \text{En}_{\text{uc}}(q)} \text{Post}(q, u) \cup \bigcup_{c \in \sigma(q)} \text{Post}(q, c). \quad (8)$$

The environment chooses any uncontrollable and any permitted-event outcome. An uncontrollable self-loop blocks updates by Equation (NP) and permits a losing infinite run. The returned policy retains one controllable at a quiescent state. For fixed goals, disabling controllables in the presence of uncontrollables preserves winning decisions (Supplement S2); quiescent handover also restricts targets.

Let $\text{Reach}_\sigma(Q_0)$ be the least set containing Q_0 and closed under $\text{Succ}_\sigma(q)$ from each non-goal state q . For each goal reachable under the returned policy, the synthesizer selects one witness for Equation (6) and returns a *handover map* $\kappa : Q_{\text{goal}} \cap \text{Reach}_\sigma(Q_0) \rightarrow Z_{\text{load}}$. Each $\kappa(q)$ satisfies $q \sim \kappa(q)$ and identifies the post-update controller state to request at load.

Runs, winning, and witnesses. A run under the policy is $q_0 a_0 q_1 a_1 \dots$, where each $q_{j+1} \in \text{Post}(q_j, a_j)$ and $a_j \in \text{En}_{\text{uc}}(q_j) \cup \sigma(q_j)$, and it stops at a goal. A run is *maximal* if it is infinite or if it is finite and ends at a goal or at a state with no successor. A single policy σ is winning for an FG-DUCS instance if, from every $q_0 \in Q_0$, (i) every reachable update state is safe, (ii) every reachable non-goal state has a successor, and (iii) every maximal update run reaches Q_{goal} after finitely many events. This *finite completion* counts observed transitions and assumes no fairness. Remaining forever at UC-enabled states loses: the environment can prevent both pending commands and quiescent handover.

A *progress-ranking function* for a winning policy is $\text{rk} : \text{Reach}_\sigma(Q_0) \rightarrow \mathbb{N}$, with value 0 at goal states and satisfying $\text{rk}(q') < \text{rk}(q)$ for every non-goal state q and every $q' \in \text{Succ}_\sigma(q)$.

FG-DUCS decides whether one policy wins from all of Q_0 and returns $(\sigma, \text{rk}, \kappa)$ or a losing-region certificate.

Definition 3.3 (Observation-driven realization and linking). A solution defines an orchestrator Π_σ with states $\text{Reach}_\sigma(Q_0)$ and entries $\text{Init}(z)$. It enables $\sigma(q)$ and accepts a report (a, q') exactly when $a \in \text{En}_{\text{uc}}(q) \cup \sigma(q)$ and $q' \in \text{Post}(q, a)$. Reported successor identities distinguish same-event outcomes.

The *linked model* is the disjoint union of monitored old, reachable update, and monitored new states. It uses unlabeled entry links $z \mapsto \text{Init}(z)$ at entry, update edges Succ_σ , and goal links $q \mapsto \kappa(q)$ through load, followed by CL^{new} .

Old monitoring ends before its own stop, update monitoring covers entry through goal, and new monitoring consumes its start once and continues through load. Entry/load add no Σ symbol or intervening game event.

3.3 Granularity Separation

A *global-boundary fragment* is an eligible input with $I = \{\star\}$, $e_\star^\vee = E^\vee$, and $g_\star = g$, and is embedded identically into FG-DUCS under the standard identification of product states. Let $B(\mathcal{W}_{\text{cell}})$ atomically aggregate $\mathcal{W}_{\text{cell}}$ from Section 2, with $I = \{\star\}$, $e_\star^\vee = e_A^\vee \parallel e_B^\vee$, and $g_\star = g_A \times g_B$, and replace ρ_A, ρ_B by $\rho_\star$. It preserves requirement boundaries and dependencies, and transports the endpoints, monitors, and Z_{load} through the standard product-state isomorphism. This pair isolates the decision consequence of event granularity while holding the supplied endpoints and requirement structure fixed.

THEOREM 3.4 (EMBEDDING THE GLOBAL BOUNDARY AND AN EXPLICIT FINE-GRAINED INSTANCE). *Every global-boundary fragment can be embedded into FG-DUCS by a game isomorphism. $\mathcal{W}_{\text{cell}}$ is winning in FG-DUCS whereas $B(\mathcal{W}_{\text{cell}})$ is losing. The pair fixes the endpoints and requirement structure and changes the granularity of transfer events.*

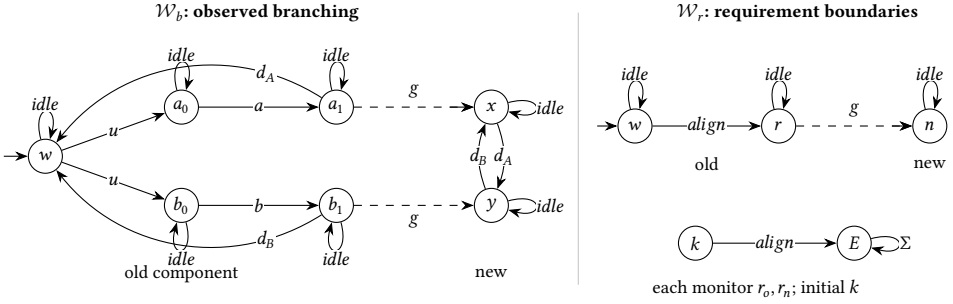
PROOF. For the embedding, map each global component state to its product tuple and carry the monitor states and pending operations unchanged. This bijection preserves ordinary synchronization, the supplied transfer relation, and stop/start initialization. Enabled uncontrollables coincide, so the non-preemption guard, safety, roots, and quiescent target matches coincide. These facts give the required game isomorphism.

For $\mathcal{W}_{\text{cell}}$, the two paths in Figure 1 cover both roots. All selected buckets are singleton and there are no uncontrollables. Their displayed monitor states are non-error; their union assigns one

action at each non-goal. Remaining lengths $6, \dots, 0$ and $5, \dots, 0$ strictly decrease to the compatible targets z_B, z_A . Thus the one policy wins from every root.

In the bulk variant, each move preserves exactly one product, inspection preserves its position, and requirement boundaries leave components unchanged. Both stations therefore cannot be empty before the first transfer. Since $g_A \times g_B$ requires precisely that state, the bulk transfer is never enabled. It stays pending on every run, so no goal is reachable. Every maximal run either ends outside a goal or is infinite; either violates finite completion. Supplement S1 additionally tabulates the full closure, including unsafe off-policy outcomes. $\square$

Figure 2 gives the remaining finite witnesses; their parameters are in Supplement S1.4.



Only u is uncontrollable in $\mathcal{W}_b$; all events are controllable in $\mathcal{W}_r$. Arrows entering states mark local initials; dashed edges are g . Unlisted plant/controller transitions are empty, unlisted non-error monitor transitions self-loop, and E absorbs Σ . In $\mathcal{W}_b$, one-state C^o/C^n loop on all six ordinary labels $\{idle, d_A, d_B\}$; requirements are empty. In $\mathcal{W}_r$, one-state C^o has alphabet $\{idle, align\}$ but only an $idle$ loop, and C^n has only that loop; both monitors start at k . Supplement S1.4 gives the complete remaining contract parameters.

Fig. 2. Finite witnesses for observation-dependent control and requirement boundaries. All ordinary transitions are shown; both contracts use the four Post rules.

PROPOSITION 3.5 (OBSERVATION AND REQUIREMENT BOUNDARIES). $\mathcal{W}_b$ has an all-root winning policy, but no fixed finite list of controllable events, whose head advances only when that event occurs and whose position is unchanged by uncontrollables, wins from w . $\mathcal{W}_r$ has a winning policy, but loses when its stop/start boundaries must form a contiguous block with no ordinary event between them; simultaneous stop/start also loses.

PROOF. In $\mathcal{W}_b$, wait for u , choose a or b from the observed successor, then transfer; roots a_1, b_1 transfer immediately. The ranks of old $w, a_0/b_0, a_1/b_1$ are 3, 2, 1. After u , the same fixed-list head faces both branches. Deleting finite $idle$ prefixes leaves a head enabled in at most one branch; an infinite $idle$ sequence cannot finish. In $\mathcal{W}_r$, $t; align; s; \rho$ reaches the goal with ranks 4, 3, 2, 1, 0. A safe $align$ must occur after t and before s , so either boundary restriction excludes every completion. Supplement S1 gives the full obstruction arguments and the executable-suite correspondence. $\square$

4 Certifying On-the-Fly Synthesis

Lazy exploration solves the implicit game and produces evidence covering every entry and retained outcome, including the requirement intervals and handover targets.

4.1 Lazy Successor Generation

FG-DUCS-OTF instantiates the reachability attractor [10] over typed Post with on-the-fly exploration [42]. It queries reached-state successors in uncontrollable-first order, with lazy controllable queries, WIN serialization and exhaustive LOSS return.

The measured candidate/worklist orders are fixed in Supplement S2. The finite candidate set $\text{Ev}(q) \subseteq \Sigma$ is the union of local events of the active components and update events in P whose precedence conditions are satisfied. If $\text{Post}(q, a) \neq \emptyset$, then $a \in \text{Ev}(q)$; write its uncontrollable and controllable parts as $\text{Ev}_{\text{uc}}(q), \text{Ev}_c(q)$. A Post query determines contract enabledness; physical executability remains an input premise.

The algorithm maintains a discovered set V , an expanded set $X \subseteq V$, and a partial table

$$\mathcal{D} : \mathcal{Q} \times \Sigma \rightarrow 2^{\mathcal{Q}} \setminus \{\emptyset\}. \quad (9)$$

Each entry satisfies $\mathcal{D}(q, a) = \text{Post}(q, a)$. For every $q \in X$, all events in $\text{Ev}_{\text{uc}}(q)$ have been queried, whereas controllable events may remain unqueried. If no uncontrollable event is enabled, the algorithm removes and queries the first element of the fixed ordering $\text{Cand}(q)$ of the unqueried $\text{Ev}_c(q)$; when the result is nonempty, it stores all successors and adds them to V . At all times, $\text{dom } \mathcal{D} \subseteq X \times \Sigma$ and $\bigcup \text{ran } \mathcal{D} \subseteq V$.

In the current partial table, which stores only successors enumerated by Post, let the known uncontrollable results be

$$U_{\mathcal{D}}(q) = \bigcup_{\substack{u \in \Sigma_{\text{uc}} \\ (q, u) \in \text{dom } \mathcal{D}}} \mathcal{D}(q, u).$$

The winning layers of the current partial game are

$$W_0 = V \cap Q_{\text{goal}}, \quad (10)$$

$$\begin{aligned} W_{j+1} = W_j \cup \{q \in X \setminus W_j \mid \text{Safe}(q) \wedge [U_{\mathcal{D}}(q) \neq \emptyset \wedge U_{\mathcal{D}}(q) \subseteq W_j] \\ \vee U_{\mathcal{D}}(q) = \emptyset \wedge \exists c \in \Sigma_c : (q, c) \in \text{dom } \mathcal{D} \wedge \mathcal{D}(q, c) \subseteq W_j\}. \end{aligned} \quad (11)$$

Let $W = \bigcup_{j \geq 0} W_j$, and define the progress-ranking function $\text{rk}(q)$ as the index of the first layer containing q . For each $q \in W \setminus Q_{\text{goal}}$ such that $U_{\mathcal{D}}(q) = \emptyset$, fix one controllable event c_q satisfying $\mathcal{D}(q, c_q) \subseteq W_{\text{rk}(q)-1}$, and define, over all $\mathcal{Q} \setminus Q_{\text{goal}}$,

$$\sigma_W(q) = \begin{cases} \{c_q\}, & q \in W \setminus Q_{\text{goal}} \wedge U_{\mathcal{D}}(q) = \emptyset, \\ \emptyset, & \text{otherwise} \end{cases} \quad (12)$$

Thus, the policy is the fail-closed empty set at uncontrollable branches and outside W , and the implementation need serialize only nonempty entries.

Exploration invariants. At each loop boundary: (I1) every $q \in X$ has all uncontrollable candidates queried, including empty buckets; (I2) at LOSS return, every safe non-goal $q \in V \setminus W$ with no enabled uncontrollable has all controllable candidates queried; (I3) promotion to W retains complete successor buckets and fixed lower-rank witnesses, which remain valid as $\mathcal{D}$ grows; and (I4) every discovered safe non-goal state outside X remains in *Open*. Expansion completes UC queries. A retry is removed only on cursor exhaustion or promotion; added buckets cannot change retained witnesses. First discovery queues safe non-goals, and only expansion removes them. These operations preserve I1–I4; propagation is drained before either return. Supplement S2 details the worklists. Bucket counters and reverse incidences implement Equation (11); $n_{\text{left}} = |Q_0 \setminus W|$ counts uncovered roots. Duplicate-free queues and one-way cursors avoid repeated scans (Supplement S2).

4.2 Complexity and Static Encoding

Let $N = |\mathcal{Q}|$, $A = \sum_q |\text{Ev}(q)|$ and $B = \sum_{q,a} |\text{Post}(q, a)|$; indexed constant-time predicates, expected-amortized constant-time dictionaries, a fixed candidate order and $O(1 + |\text{Post}(q, a)|)$ queries give $O(N + A + B)$ time and space after endpoint/eligibility preprocessing (Supplement S2). These

Algorithm 1: FG-DUCS-OTF

Input: Typed update contract (Definition 3.1)

Output: (σ, rk, κ) or (no solution, L)

```

1  $V \leftarrow Q_0$ ;  $X, \mathcal{D}, \text{Retry} \leftarrow \emptyset$ ;  $W \leftarrow V \cap Q_{\text{goal}}$ ;  $\eta_{\text{left}} \leftarrow |Q_0 \setminus W|$ ;
2  $\text{Open} \leftarrow \{q \in Q_0 \mid \text{Safe}(q) \wedge q \notin Q_{\text{goal}}\}$ ; set  $rk(q) \leftarrow 0$  for  $q \in W$ ;
3 while  $\text{true}$  do
4   Drain the reverse-edge worklist, updating bucket counters,  $(W, rk)$ ,  $\eta_{\text{left}}$ , and  $\text{Retry}$ ;
5   if  $\eta_{\text{left}} = 0$  then return  $(\sigma, rk|_{\text{Reach}_\sigma(Q_0)}, \kappa)$  constructed from Equations (6) and (12);
6   if  $\text{Open} \neq \emptyset$  then
7     Remove  $q$  from  $\text{Open}$ , add it to  $X$ , and query all uncontrollable candidates;
8     if a nonempty uncontrollable successor set exists then
9        $\text{Cand}(q) \leftarrow \emptyset$ ;
10    else
11      Set  $\text{Cand}(q)$  to the fixed-order controllable cursor; query to the next nonempty bucket or
12      exhaustion;
13    Insert  $q$  into  $\text{Retry}$  iff  $q \notin W$  and  $\text{Cand}(q) \neq \emptyset$ ;
14  else
15    if  $\text{Retry} = \emptyset$  then return (no solution,  $L \leftarrow V \setminus W$ );
16    Remove  $q$  from  $\text{Retry}$ ; query to the next nonempty bucket or exhaustion; reinsert iff  $q \notin W$  and
17     $\text{Cand}(q) \neq \emptyset$ ;
18  Enqueue each newly discovered goal for its sole  $W$  insertion with  $rk = 0$ , or each safe non-goal in  $\text{Open}$ ;
```

are explicit-game bounds: compact inputs and all-entry sets can be exponential. Java's sorting, repeated root checks and transition scans are included in the measurements; Direct-Full materializes the identical roots and terminalized game before solving it.

LEMMA 4.1 (WINNING LAYERS AND A SPOILING SET). *The set W of Equations (10)–(11) admits the policy of Equation (12) and a layer rank that decreases along every successor. When no solution exists, put $L = V \setminus W$. For each safe $q \in L$, if an uncontrollable is enabled, one of its outcomes remains in L ; otherwise, one outcome of every model-enabled controllable remains in L . Consequently, every state in $Q_0 \cap L$ is losing.*

PROOF. Induction over layers shows that either all uncontrollable successors or all outcomes of the selected c_q enter the preceding layer. On failure, all safe non-goal states and all necessary candidates have been expanded; if none of the outcomes above remained in L , Equation (11) would add q to W . An environment policy selecting a remaining outcome either reaches an unsafe state or a deadlock, or proceeds forever inside finite $L \setminus Q_{\text{goal}}$; each case contradicts finite reachability. $\square$

LEMMA 4.2 (FINITE COMPLETION AND RANK). *Fix a policy on a finite reachable graph, with goals terminal, safe reachable states, and nonempty successors outside Q_{goal} . The following are equivalent: (i) every maximal run reaches Q_{goal} ; (ii) the reachable non-goal subgraph is acyclic; (iii) a natural-number rank, zero at goals and positive elsewhere, strictly decreases on every edge from a non-goal. The nonempty-successor premise excludes non-goal sinks, which would satisfy acyclicity alone.*

PROOF. A reachable non-goal cycle admits an infinite run. Without such a cycle, every run has at most as many non-goal steps as there are non-goal vertices; nonempty successors force it to end at a goal. The longest remaining path to a goal gives a rank, and strict rank decrease excludes every non-goal cycle. $\square$

THEOREM 4.3 (TERMINATION, SOUNDNESS, AND COMPLETENESS). *For every eligible finite FG-DUCS instance with Equation (NP) and quiescent goals in Equation (6), Algorithm 1 terminates. A returned*

($\sigma, \text{rk}, \kappa$) satisfies every declared interval's safety relative to the supplied monitor/residual semantics and externally justified RS for ι , preserves every uncontrollable event and every outcome of each policy event, reaches Q_{goal} finitely without deadlock, and gives each reached goal a typed-game-relative compatible target in Z_{load} . It returns no solution iff no winning policy covers Q_0 in that supplied game.

PROOF. Termination. The finite input induces finitely many states and candidates. Each state is discovered and expanded at most once; each retry consumes a new candidate, and each promotion adds a state to W once. Reverse incidences are processed on those promotions, so every worklist drain is finite. Eventually all roots win or both exploration queues empty.

Soundness. Induction on winning layers gives complete retained buckets whose successors have smaller ranks: I1 supplies all uncontrollables, or the selected nonempty controllable bucket supplies all outcomes. I3 preserves this evidence. If all roots win, their policy closure is safe, has successors at every non-goal, and satisfies Lemma 4.2. Goal recognition uses physical quiescence and supplies each compatible $\kappa(q)$.

For interval safety, entry inherits old monitor states, initializes update monitors, and leaves new monitors inactive. An ordinary step advances exactly the active monitors. Transfer additionally changes one version and consumes its operation; stop removes its monitor before observation; start installs its supplied residual after the boundary while advancing the previously active monitors. Inactive entries remain $\perp$, and pending operations enforce one-shot execution. Induction preserves the version and activity invariants. Every retained state is non-error, so each suffix is safe for its initialized monitor; RS extends this safety to its supplied activation history. Goal matching preserves the component and new-monitor projection into the verified endpoint.

Completeness. At failure, a root lies in $L = V \setminus W$, and L contains no goal. I4 ensures every safe non-goal in L was expanded. With uncontrollables, I1 and exhausted propagation imply an uncontrollable outcome in L ; otherwise promotion would apply. Without uncontrollables, I2 gives every controllable bucket, each with an outcome in L for the same reason. Against any policy, the environment keeps choosing these outcomes; permitting no event produces a deadlock. The resulting run reaches an unsafe state, deadlocks outside a goal, or remains in L forever. Each contradicts winning. Hence a winning policy rules out failure; termination then forces success. Supplement S2–S3 adds implementation-cost details and an expanded interval derivation. $\square$

COROLLARY 4.4 (CERTIFICATE CHECKS). Let $R_\sigma := \text{Reach}_\sigma(Q_0)$. A WIN certificate checks $Q_0 \subseteq R_\sigma = \text{dom rk}$, safety on R_σ , and, for each $q \in R_\sigma \setminus Q_{\text{goal}}$, $\sigma(q) \subseteq \text{En}_c(q)$, $\emptyset \neq \text{Succ}_\sigma(q) \subseteq R_\sigma$, and rank decrease for every successor. It checks $\text{dom } \kappa = R_\sigma \cap Q_{\text{goal}}$ and, there, rank zero, $U(q) = \emptyset$, $\kappa(q) \in Z_{\text{load}}$, and $q \sim \kappa(q)$; unserialized σ entries are empty. A LOSS certificate checks $L \subseteq Q$, $Q_0 \cap L \neq \emptyset$, and $L \cap Q_{\text{goal}} = \emptyset$. Each safe $q \in L$ has an uncontrollable outcome in L if one is enabled; otherwise every enabled controllable has an outcome in L . Unsafe states and non-goal deadlocks lose. These checks certify search results relative to shared Post and endpoint predicates [23]; Section 5 states the trust boundary.

Static reachability encoding. The environment chooses roots, uncontrollables and outcomes; otherwise the controller chooses an event. Splitting nonempty buckets through outcome vertices gives an equivalent strong-plan game in $O(N + A + B)$. Supplement S2 proves this encoding and preservation under complete successor isomorphism; κ remains typed endpoint evidence.

4.3 A Separation in Successor Generation

LEMMA 4.5 (CONTAINMENT IN FULL EXPANSION). *Let both methods use identical Q_0 , candidates, exact Post, and terminal predicates, expanding neither unsafe states nor goals. Every state discovered and every bucket queried by OTF is respectively discovered and queried by Direct-Full.*

The discovery-order induction, including terminal successors and query sources, is given in Supplement S2.

THEOREM 4.6 (INDEPENDENT UPDATES). *For each $K \geq 1$, there is an eligible FG-DUCS instance with K independent component transfers such that Algorithm 1, choosing a pending transfer before ordinary events, discovers exactly $K + 1$ update states, whereas Direct-Full discovers 2^K states.*

PROOF. Each component has one old and one new state, related by a singleton transfer. Every version has a controllable *tick* self-loop. The old and new controllers admit *tick*, so both endpoints are deadlock-free. There are no monitors, precedence constraints, or uncontrollables; the unique new endpoint is loadable. A state is identified by the subset S of transferred components; ρ_i adds $i \notin S$ and the goal is $S = I$. Every subset is reachable by performing exactly its transfers, giving 2^K states. Lazy expansion queries one pending transfer first at each non-goal. Serving *Open* before *Retry* follows one chain to the goal after K steps; the next worklist drain propagates ranks $0, \dots, K$ to the sole root before any retry. Exactly the chain's $K + 1$ states are discovered. Both methods use the same quiescent terminal goal. The ordinary self-loops remain losing choices under finite completion. $\square$

5 Implementation

Our MTSA extension [16] implements canonical configurations, lazy buckets and certifying checks for WIN and LOSS [32].

TCB 1—shared model semantics: the certificate checker shares the parser, Post, monitors, transfer rules, and endpoint/goal code with the synthesizer.

TCB 2—checked search result: it checks roots, safety, all outcomes, ranks/traps and handover, validating search control relative to shared semantics.

TCB 3—cross-checks: the explicit checker constructs successors separately but shares the frontend, endpoints and goal predicate. The restricted raw-LTS oracle also parses independently. Both were revised by the same author team. Residual meanings and runtime conformance remain external premises.

Use of generative AI. OpenAI Codex (GPT-6 Astra; start date [TO CONFIRM]) assisted contract design, semantic analysis, Java of f implementation/tests, campaign, rendering and analysis scripts, and figures. June–August 2026 implementation and ICSE-format drafting also used generative AI [TO CONFIRM: systems/versions]. From September 14, Claude Fable 5.1 in the Anthropic desktop app assisted planning/structural diagnosis, authored the Windows driver and runbook, inspected returned raw, read source code to diagnose legacy comparisons, and drafted author decisions and task instructions. Claude authored neither manuscript prose, Java nor the renderer. ChatGPT reviewed manuscript PDFs on August 7 and September 18 [TO CONFIRM: models]. Tests, raw-LTS oracles, certificate checks, primary sources and author review checked these contributions; shared authorship and AI assistance permit correlated faults. Models/logs/raw were processed locally; PDFs went to external review services. Numerical evidence uses saved outputs, not initial AI readings. Separately, Codex assisted prose drafting/revision; authors retain responsibility.

Runtime adapter. A platform interprets the returned policy/rank/handover table under A1–A4. Java names entry/load hotSwapIn/hotSwapOut; the linked LTS uses an atomic goal quotient. A phase-disjoint observation map α must satisfy:

A1 (endpoints): actual endpoint observations map to same-labeled edges of $CL^\vee$ and preserve its component and monitor states. **A2 (update observations and progress):** at a non-goal, filtering the complete plant event set removes only controllable events outside $\sigma(q)$; every resulting event/outcome maps into $\text{Post}(q, a)$ and is reported with its successor identity. The resulting set is nonempty and a next event is eventually observed. The latter is an execution-progress premise, independent of fairness among event choices. **A3 (commands and observation):** commands are issued only at uncontrollable-quiescent states, finish atomically and failure-free in finite time, and report an outcome included in the supplied transfer or monitor-lifecycle rule. Internal τ steps, unobservable update outcomes, and divergence inside an update operation are outside the model: A3 requires one terminating observed command transition with its successor identity. **A4 (boundaries):** entry atomically observes the actual reachable old state z and switches to $\text{Init}(z)$; at a quiescent goal, ordinary controllables are disabled and load loads $\kappa(q)$ and switches control. Each boundary is a finite, failure-free linearizable operation preserving the specified component and monitor states [21].

THEOREM 5.1 (CONDITIONAL TRACE LIFTING). *For an eligible contract and a returned $(\sigma, \text{rk}, \kappa)$, every execution satisfying A1–A4 lifts through α to the linked model. Starting at $\alpha(x_0) = \text{Init}(z) \in Q_0$, its update segment reaches a quiescent goal within $\text{rk}(\text{Init}(z))$ observed update-game events and then loads the compatible endpoint through load. Each declared active interval satisfies its supplied monitor/residual semantics, conditional on RS for the supplied initialization maps ι_r . Correctness of those meanings with respect to an application’s intended requirements is an input obligation.*

PROOF. A1 maps the old prefix to the monitored endpoint. A4 atomically maps its entry state z to $\text{Init}(z) \in Q_0$, so all-root coverage places it in the certified policy domain. Induct on observed update events. At a non-goal, A2 preserves all uncontrollables and allows only policy controllables; each reported outcome lies in its complete Post bucket. For a command, A3 supplies one finite observed transition at a quiescent source. Thus the next mapped state is a policy successor, remains in the safe certified domain, and has strictly smaller rank.

A2 supplies a next event eventually at every non-goal; neither infinite stalling nor a non-goal deadlock satisfies that premise. A natural-number rank initially r permits at most r decreases, forcing a goal within r observed events. There, quiescence excludes an uncontrollable competitor and A4 disables ordinary controllables and performs the finite load of $\kappa(q)$. A1 maps the new continuation to its verified endpoint. The interval induction and RS in Theorem 4.3 give the stated relative safety across these boundaries. Supplement S3 expands the observation and boundary correspondence. $\square$

6 Evaluation

We evaluate **RQ1** correctness, **RQ2** capability boundaries, **RQ3** synthesis cost on application models, and **RQ4** scaling with updates, branching and requirements.

Constructing the benchmark inputs. Nine maintained models manually adapt eight DUCS families. They declare old/new component pairs and local transfers, mainly between corresponding named states. Production Cell also maps polished states to an arm state; Railcab permits two milestone outcomes; PowerPlant leaves its initial state unmapped. The frontend generates individual stop/start events; new monitors use fluent lookup, while update monitors use a fixed post-entry state (S3). All reachable new-closed-loop states are load candidates, filtered by the goal test. No

explicit precedence edges are declared (contract coverage in S4): R2 encodes selected ordering through update-time monitors; R1 monitors boundary fluents and bans specified actions. Base adds no transition requirement. Under activation-safe A, all 138/138 NEW initializers satisfy RS inclusion; equality holds when actual safe histories are nonempty. Under entry-scoped E, 89/135 UPD initializers are exact by the update-stage-fluent condition (S3, Lemma E'). The remaining 46 reference action predicates and remain unverified. H admits past violations and supplies no benchmark RS guarantee; all strict-H diagnostics remain in S4.

Design and measures. RQ3 retains all 27 instances (nine models, base/R1/R2), comparing lazy, eager-controllable, update-first, *Direct-Full* and *Legacy*, this fork's legacy DUCS path on the same fine-grained inputs. Legacy retains its GR objective without endpoint handover; Direct-Full uses exactly our Post/goal. On 8 original DUCS models and 21 monolithic conditions, a lab-maintained build of the original DUCS source returns 21 controllers; the aligned fork agrees on all 16 jointly processed conditions, with 15 matching first-trial sizes and a 5-state Railcab difference amid repetition variation (Supplement S4). The remaining 5 stop before a legacy GR decision (3 parser, 2 instrumentation), providing no evidence of legacy synthesis failure. Capability separations use RQ2 witnesses.

Trials run sequentially on Windows 10, Xeon W-2265, 256 GB RAM, Temurin 17.0.20.1, with a 64 GiB heap and 1,200 s limit. Stage 1 covers all 135 RQ3 pairs. Valid decisions receive four further runs; failures are retained without repetition. Medians and min-max require five valid, consistent completions. An authorized same-setting Workflow/R2 Direct-Full supplement times out at 1200 s; the original operational interruption is retained in S4. A separate 3600 s PC Arms=2/R2 FG trial also times out and contributes no median.

We record states, queries, transitions, solver time, sampled peak JVM RSS and policy size. LTS time includes checking; hub is solve-only; Legacy uses its native timer. Endpoint/process times are separate. Completed-pair ratios use solver intervals, excluding frontend/startup; S4 also reports total JVM elapsed time. Mac pilot times are excluded.

6.1 RQ1: Correctness

Answer. All 92 jobs agree with independently derived raw-LTS expectations (Table 1). Cases cover semantic boundaries, related variants and endpoint contracts, including missing controller events, deadlocks and suppressed uncontrollables (S4).

Table 1. Local validation of the confirmed semantics, separate from Xeon cost measurements. Counts are jobs; variants share source models.

Stratum	Jobs	WIN	LOSS	Invalid	Expected agreement
Semantic models, two methods	82	34	48	0	82/82
Endpoint contracts, two methods	10	2	0	8	10/10
RQ1 total	92	36	48	8	92/92

The Python oracle independently parses restricted FSP and derives synchronous graphs, ranks and losing closure. Separate probes confirm transfer-blocking old UC loops, new UC loops changing WIN to LOSS, and quiescence after a finite UC suffix.

All eligible jobs pass certificates and separate exhaustive checks; every WIN passes atomic Link. Section 5 states the trust boundary; S4 gives denominators and test counts.

6.2 RQ2: Capability

Answer. Fine-grained policies win where each tested restriction loses: eight LTS files yield ten WIN and four LOSS, all matching independent expectations (Table 2). They test the cell obstruction, both fixed heads, contiguous boundaries and neutral controls (S1). Industry/R1’s checked LOSS rejects its supplied contract; it is not a capability separation.

Table 2. Completed formal capability checks. W/L counts winning/losing jobs. Both fixed resolving heads are tested.

Witness	Policy/model	Separation W/L	Neutral W/L
Production Cell	Local transfer	1/0	1/0
	Simultaneous transfer	0/1	1/0
Observed branch	Full policy	1/0	1/0
	Fixed heads	0/2	2/0
Requirement boundary	Interleaved	1/0	1/0
	Contiguous block	0/1	1/0

6.3 RQ3: Synthesis Cost

Answer. Within the fixed budget, lazy decides more instances; its completed Direct-Full pairs agree and are faster. Table 3 retains all 135 cells. Lazy synthesis returns 25 WIN, one LOSS (Industry/R1), and one TO (PC Arms=2/R2). Direct-Full returns 14 WIN and 13 TO, including the marked supplement. Their 14 completed decisions agree; Direct-Full/FG ratios of five-run median solver times range from 1.58 to 2,123.81, with median 9.14. These are paired costs, not realizability separations. For 12 DF timeouts with completed FG runs, censored JVM-attempt/FG elapsed-median lower bounds span 2.12–910.06 (S4); these bounds cover total time, not solver time. PC Arms=2/R2 remains unresolved after 3600 s, with 67.1 GiB peak process RSS under the 64 GiB heap cap.

Candidate orders and Open/Retry ties are fixed in S2; eager and update-first test sensitivity to order and materialization. Eager-controllable decides 17 instances; update-first decides 24 (23 WIN, one LOSS), versus lazy’s 26. All paired decisions agree; update-first is faster on 8/24 pairs. Legacy has nine WIN agreeing with FG, 13 OOM and five N/M (capture cap). The author-side cap prevents measurement before synthesis, evidencing no Legacy failure. Its GR objective and native timer delimit this reference comparison.

Industry/R1 loses at 42 of 131 entries: the FG contract requires every entry to force quiescent handover to a verified new-controller state. Separate exhaustive checking confirms the losing region: after all nine updates, an incompatible state admits repeated DSD1 rejection, while R1 prevents earlier history repair (Supplement S4). By contrast, a lab-maintained build of the original DUCS source returns a 2,312-state monolithic controller under T_Empty for the source model of the fine-grained adaptation, as does the fork once entry labels are aligned. Published DUCS continues control under the new specification without requiring a preverified new-controller connection [34, Defs. 4.1–4.3]. This LOSS is specific to the supplied FG contract.

Exempting only validateGF1 from the three R1 bans makes all 131 roots WIN with certificate, Link and exhaustive checks, keeping other inputs and Post/goal fixed. This Mac contract variant assumes validation is acceptable during update and is excluded from Table 3 (S4).

6.4 RQ4: Scaling

Answer. Lazy exploration reduces cost along the independent-update and tested hub axes, while Travel retains a shared concurrency limit. All four methods finish independent updates for $K =$

Table 3. RQ3 solver time (seconds), all 27 instances and five methods. Medians require five valid, consistent completions; other cells retain statuses. PC Arms=1 is the DUCS Production Cell benchmark; Arms=2 is its extension included in the 27 instances since the ICSE draft. Ranges and full metrics: Supplement S4.

Model	Req.	FG lazy	Eager	Update-first	Direct-Full	Legacy
GSM	BASE	W 0.0224 [†]	W 0.0332 [†]	W 0.0233	W 0.0416	W 0.241
	R1	W 0.0228	W 0.0348 [†]	W 0.0243	W 0.045	W 0.306
	R2	W 0.0201 [†]	W 0.0281	W 0.0225	W 0.0327 [†]	W 0.25
Industry	BASE	W 0.19	W 14.5	W 0.542	W 154	N/M
	R1	L 562	TO	L 461 [†]	TO	N/M
	R2	W 1.15	W 2.96	W 0.455	W 28	N/M
MetaSocket	BASE	W 0.0149 [†]	W 0.0302 [†]	W 0.0165 [†]	W 0.0408	W 0.323
	R1	W 0.0136 [†]	W 0.0278 [†]	W 0.0147	W 0.0426 [†]	W 0.443
	R2	W 0.0152 [†]	W 0.018	W 0.0154 [†]	W 0.024 [†]	W 0.29
PowerPlant	BASE	W 0.0463 [†]	W 0.487	W 0.052	W 1.85	W 18 [†]
	R1	W 0.125 [†]	W 1.08	W 0.152	W 5.51	W 896 [†]
	R2	W 0.18	W 0.251	W 0.114	W 1.01	W 8.25
PC Arms=1	BASE	W 0.0741 [†]	W 26	W 0.886	W 157	N/M
	R1	W 0.0793	W 188	W 3.15	TO	OOM
	R2	W 6.16 [†]	W 12.2	W 1.17	W 77.8	N/M
PC Arms=2	BASE	W 2.61 [†]	TO	TO	TO	OOM
	R1	W 2.6	TO	TO	TO	OOM
	R2	TO ^a	TO	TO	TO	OOM
Railcab	BASE	W 0.169	TO	W 2.26	TO	OOM
	R1	W 3.18	TO	W 6.74	TO	OOM
	R2	W 15.5	W 38.7	W 4.16	W 283	OOM
Surveillance	BASE	W 2.01	TO	W 2.63 [†]	TO	OOM
	R1	W 18.6	TO	W 126	TO	OOM
	R2	W 37.1	TO	W 16.1	TO	OOM
Workflow	BASE	W 0.111	W 508	W 0.116	TO	OOM
	R1	W 0.325	TO	W 0.215	TO	OOM
	R2	W 16.1	W 23.9	W 3.18	TO [‡]	OOM

W/L: winning/losing. Seconds: median only for five valid, consistent completions; ranges in S4. TO: 1,200 s timeout; OOM: out of memory.

N/M (capture cap): author-side M9MtsSnapshot instrumentation stops before synthesis at its fixed 250,000-state cap; no evidence of Legacy capability or solver failure. Original operational interruptions are retained in S4. Legacy uses native semantics. Other incomplete groups show statuses, without medians. [†]: (max – min)/median > 10% (also a zero median with a positive range); see Supplement S4.

^a The separate 3,600 s single trial also timed out (peak process RSS 67.1 GiB; 64 GiB heap cap). [‡] The original trial was operationally interrupted; the pre-authorized same-setting replacement timed out at 1,200 s. Both records remain in Supplement S4. RSS does not establish heap saturation.

2, ..., 20 (Figure 3). At $K = 20$, lazy and update-first discover 21 states, eager 211, and Direct-Full 1,048,576. Their five-run median times are respectively 0.0354, 0.0318, 0.0625 and 853 s. The exact separation concerns this constructed family.

Across 33 controlled instances (132 single-run jobs), every method returns 31 WIN and two LOSS. With eight monitors, lazy discovers 18 states versus Direct-Full's 1,179,640; larger transfer fanout forces every method to retain all outcomes (S4; no timing medians).

Travel retains all 48 (N, K) instances in Figure 3. Lazy, eager and update-first each finish 20; Direct-Full finishes 18. On 18 completed pairs, lazy uses 3.4–46.2% of Direct-Full's states; median-time ratios are 1.49–34.21 (median 3.93). Lazy and update-first match states, queries and outcomes on all 20 completions. All methods remain unresolved at $N = 4$, $K \geq 8$ and every tested $N \geq 6$ (1,200 s/64 GiB): lazy exploration benefits the K axis but leaves a shared concurrency limit.

All 90 hub instances (30 seeds, three UC profiles, $n = 4$) finish five repetitions per method with agreeing decisions. Direct-Full/lazy ratios are 5.35–118.38 (median 78.11); lazy's largest median is 0.918 s. Each seed/profile is plotted separately.

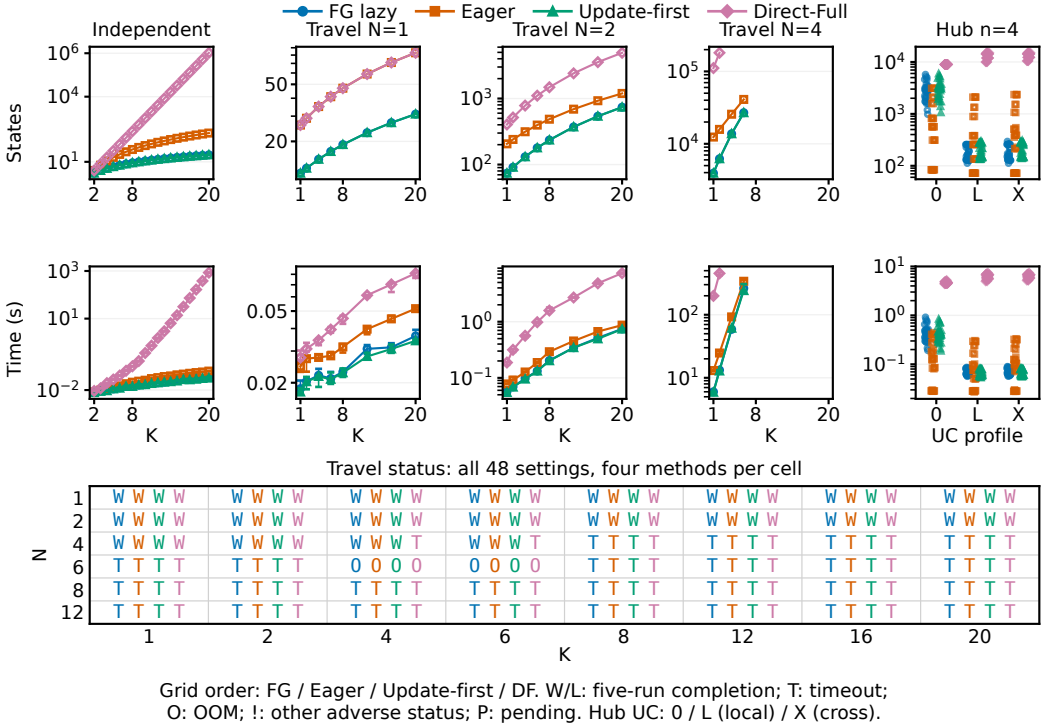


Fig. 3. RQ4 first-trial discovered states (top) and five-run median solver time with min-max (middle). Hub uses solve-only time, plotting every seed/profile. The bottom grid retains all 48 Travel settings and four methods, including TO/OOM; incomplete cells have no timing median. Full panels: Supplement S4.

6.5 Threats to Validity

Checkers, tests and code share the author team and AI assistance, allowing correlated faults. The raw oracle supports a restricted language. A establishes RS inclusion for all 138/138 NEW initializers; equality needs nonempty actual safe histories, whose plant-level nonemptiness is unverified. E certifies 89/135 UPD initializers; the remaining 46 reference action predicates and need entry-value-dependent language checks. H supplies no benchmark RS guarantee. A1–A4 and runtime loading need validation. Nine adaptations of eight families are correlated cases, not independent samples; interpretation is by family. Constructed witnesses limit generalization. Legacy denotes our fork; its GR objective differs from FG. It does not process 5/21 original inputs accepted by the lab build; one first-trial size differs, and sizes vary across repetitions (Supplement S4).

Timings select completed pairs; all 135 cells retain unresolved outcomes. The author-side capture cap prevents measurement, implying no Legacy failure. Update-first wins some timing pairs. Industry/R1's LOSS and validation-permitting WIN depend on the contract. Sampled RSS can miss peaks. One machine, heap and synthetic families limit generalization; all methods remain unresolved on Travel at every tested $N \geq 6$. PC Arms=2/R2 remains unresolved at 3600 s; S4 preserves both original and supplementary records. Earlier adverse results and old-goal checks remain archived outside current measurements.

7 Related Work

FG-DUCS inherits established reachability attractors and OTF exploration [10, 11, 42], the DUCS update-control problem [34], and trace-dependent residual obligations from Live Synthesis [17]. FG combines nondeterministic component transfers, individual requirement boundaries and ι under RS, precedence, Z_{load} and quiescent goals, with finite completion for all entries and outcomes, no fairness, and WIN/LOSS evidence. Table 4 compares native contracts; marked-state nonblocking instead permits an existential completing continuation.

Table 4. Primary-source contracts and progress premises (UC: uncontrollable). Full comparison: Supplement S5.

Work	Update/control contract	Roots, outcomes and progress
DUCS [33, 34]	Global environment/spec switch; set-valued transfer; order formula T	All old states/outcomes; eventual update after hotSwap; legal, deadlock-free (Defs. 4.1–4.3)
GR(1) bridge [2]	Old safety before switch, new spec afterward	All prefixes meeting new environment safety; bounded switch (Def. 3.1); later liveness assumes justice
Live Synthesis [17]	Old/new LTL; residual $\Psi(\eta, \varphi)$	Universal: all old prefixes/new traces; old obligations finitely satisfied, one eventual update (Thm. 6.4)
DES state attraction [36]	Common deterministic plant; task recognizer	All requests; UC preserved; weakly invariant startup; bounded first completion (Def. 2, Thm. 1)
OTF-DCS [11]	Components, marked/illegal states, control partition	All UC; designated disturbances; eager director (Def. 3); every prefix has a marked extension (Def. 4)
Dynamic Feature [41]	Feature changes, reinitialization, configuration/behavior constraints	Mixed control; every reachable state has a jointly marked continuation (Sec. 7)

DUCS exposes global reconfiguration, whole-set stop/start and set-valued transfers; stages can be encoded in the new environment [34, Sec. 4.2, Defs. 4.1–4.2]. Individual boundaries and residuals then require explicit mode/history encoding rather than native inputs. Direct-Full measures explicit expansion of the same FG Post/goal, not a translated DUCS implementation.

The GR(1) bridge preserves old safety, without old justice, until a bounded switch under new environment safety; new justice assumes environment justice [2, Def. 3.1]. FG adds individual residuals, transfer outcomes, quiescent boundaries and all-root coverage.

DES state attraction bounds completion on a deterministic plant with a weakly UC-invariant startup set [36, Def. 2, Thm. 1]. FG instead quantifies over all roots and transfer outcomes in a mode/residual product with compatible quiescent targets.

OTF-DCS [11, Defs. 3–4] explores component-based directed control: each prefix needs a completing extension, whereas every FG continuation must complete handover.

Scenario, process, workflow and UAV updates preserve obligations [18, 30, 35, 40, 44]; evolving monitors carry history [7, 26]. Reconfiguration and lifecycle methods coordinate control, structure and replacement [6, 8, 12, 14, 24, 38]; network updates address state transfer and stages [31, 37, 39]. FG requires all-outcome completion at a verified endpoint.

8 Conclusion

FG-DUCS certifies all-root, all-outcome completion or failure relative to the supplied finite model and RS, with execution guarantees under A1–A4. A establishes RS inclusion for all 138 NEW initializers, with equality on nonempty safe histories. E certifies 89/135 UPD initializers; 46 remain unverified. Lazy synthesis decides 26/27 Xeon instances versus Direct-Full's 14, with paired speedups of 1.6–2,124 $\times$; hub yields 5.3–118 $\times$. Travel's shared resource limits, the certified LOSS and unresolved outcomes delimit the empirical results.

Data Availability. The anonymous repository [4] provides the complete MTSA-derived tool sources, FG-DUCS extensions, models, scripts, RQ1/RQ2 evidence and supplement. It also provides generated tables, inputs and diffs for Xeon RQ3, all RQ4 families, both RQ3 supplementary campaigns and the separate monolithic legacy-fidelity reference results. Larger raw outputs are packaged as two verified split assets; their publication in the v2-preview pre-release is pending. The submission version is designated fse27-submission; this tag will be created after v2 is frozen. A public HTTPS clean clone, empty-cache source build and RQ1/RQ2 derive-only checks have passed; public asset restoration awaits the Release. Included scripts fetch third-party dependencies from official upstream distributions; the shaded JAR is not distributed. Checker-pass records and serialized bodies are distinguished, and personal paths anonymized.

References

- [1] Bowen Alpern and Fred B. Schneider. 1985. Defining Liveness. *Inform. Process. Lett.* 21, 4 (1985), 181–185. doi:10.1016/0020-0190(85)90056-0
- [2] Gal Amram, Shahar Maoz, Itai Segall, and Matan Yossef. 2022. Dynamic Update for Synthesized GR(1) Controllers. In *Proceedings of the 44th International Conference on Software Engineering (ICSE '22)*. ACM, New York, NY, USA, 786–797. doi:10.1145/3510003.3510054
- [3] Jesper Andersson, Luciano Baresi, Nelly Bencomo, Rogério de Lemos, Alessandra Gorla, Paola Inverardi, and Thomas Vogel. 2013. Software Engineering Processes for Self-Adaptive Systems. In *Software Engineering for Self-Adaptive Systems II*, Rogério de Lemos, Holger Giese, Hausi A. Müller, and Mary Shaw (Eds.). Lecture Notes in Computer Science, Vol. 7475. Springer, Berlin, Germany, 51–75. doi:10.1007/978-3-642-35813-5_3
- [4] Anonymous FG-DUCS Authors. 2026. FG-DUCS Evaluation Artifact. Public anonymous replication package. Submission version: tag fse27-submission, to be created after the v2 freeze. <https://github.com/research-artifact-archive/research-artifact>
- [5] Luciano Baresi, Carlo Ghezzi, Xiaoxing Ma, and Valerio Panzica La Manna. 2017. Efficient Dynamic Updates of Distributed Components Through Version Consistency. *IEEE Transactions on Software Engineering* 43, 4 (2017), 340–358. doi:10.1109/TSE.2016.2592913
- [6] Davide Bresolin and Ivan Lanese. 2021. Static and Dynamic Property-Preserving Updates. *Information and Computation* 279 (2021), 104611. doi:10.1016/j.ic.2020.104611
- [7] Marc Carwehl, Thomas Vogel, Genáina Nunes Rodrigues, and Lars Grunske. 2023. Runtime Verification of Self-Adaptive Systems with Changing Requirements. In *18th IEEE/ACM International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS '23)*. IEEE, Piscataway, NJ, USA, 104–114. doi:10.1109/SEAMS59076.2023.00024
- [8] Maverick Chardet, Hélène Coullon, and Simon Robillard. 2021. Toward Safe and Efficient Reconfiguration with Concerto. *Science of Computer Programming* 203 (2021), 102582. doi:10.1016/j.scico.2020.102582
- [9] Betty H. C. Cheng, Rogério de Lemos, Holger Giese, Paola Inverardi, Jeff Magee, Jesper Andersson, Basil Becker, Nelly Bencomo, Yuriy Brun, Bojan Cukic, Giovanna Di Marzo Serugendo, Schahram Dustdar, Anthony Finkelstein, Cristina Gacek, Kurt Geihs, Vincenzo Grassi, Gabor Karsai, Holger M. Kienle, Jeff Kramer, Marin Litoiu, Sam Malek, Raffaella Mirandola, Hausi A. Müller, Sooyong Park, Mary Shaw, Matthias Tichy, Massimo Tivoli, Danny Weyns, and Jon Whittle. 2009. Software Engineering for Self-Adaptive Systems: A Research Roadmap. In *Software Engineering for Self-Adaptive Systems*. Lecture Notes in Computer Science, Vol. 5525. Springer, Berlin, Germany, 1–26. doi:10.1007/978-3-642-02161-9_1
- [10] Alessandro Cimatti, Marco Roveri, and Paolo Traverso. 1998. Strong Planning in Non-Deterministic Domains via Model Checking. In *Proceedings of the Fourth International Conference on Artificial Intelligence Planning Systems (AIPS '98)*. AAAI Press, Menlo Park, CA, USA, 36–43. <https://f.aaai.org/Library/AIPS/1998/aips98-005.php>
- [11] Daniel Ciolek, Matias Duran, Florencia Zanollo, Nicolás Pazos, Julián Braier, Víctor Braberman, Nicolás D'Ippolito, and Sebastián Uchitel. 2023. On-the-Fly Informed Search of Non-Blocking Directed Controllers. *Automatica* 147 (2023), 110731. doi:10.1016/j.automatica.2022.110731
- [12] Roberto Di Cosmo, Jacopo Mauro, Stefano Zacchiroli, and Gianluigi Zavattaro. 2014. Aeolus: A Component Model for the Cloud. *Information and Computation* 239 (2014), 100–121. doi:10.1016/j.ic.2014.11.002
- [13] Hélène Coullon, Ludovic Henrio, Frédéric Loulergue, and Simon Robillard. 2024. Component-Based Distributed Software Reconfiguration: A Verification-Oriented Survey. *Comput. Surveys* 56, 1, Article 2 (2024), 37 pages. doi:10.1145/3595376
- [14] Gwenaél Delaval and Éric Rutten. 2010. Reactive Model-Based Control of Reconfiguration in the Fractal Component-Based Model. In *Component-Based Software Engineering (Lecture Notes in Computer Science, Vol. 6092)*. Springer, Berlin,

- Heidelberg, 93–112. doi:10.1007/978-3-642-13238-4_6
- [15] Nicolás D'Ippolito, Victor Braberman, Jeff Kramer, Jeff Magee, Daniel Sykes, and Sebastián Uchitel. 2014. Hope for the Best, Prepare for the Worst: Multi-tier Control for Adaptive Systems. In *Proceedings of the 36th International Conference on Software Engineering (ICSE '14)*. ACM, New York, NY, USA, 688–699. doi:10.1145/2568225.2568264
- [16] Nicolas D'Ippolito, Dario Fischbein, Marsha Chechik, and Sebastian Uchitel. 2008. MTSA: The Modal Transition System Analyser. In *2008 23rd IEEE/ACM International Conference on Automated Software Engineering*. IEEE, Piscataway, NJ, USA, 475–476. doi:10.1109/ASE.2008.78
- [17] Bernd Finkbeiner, Felix Klein, and Niklas Metzger. 2022. Live Synthesis. *Innovations in Systems and Software Engineering* 18, 3 (2022), 443–454. doi:10.1007/s11334-022-00447-5
- [18] Carlo Ghezzi, Joel Greenyer, and Valerio Panzica La Manna. 2012. Synthesizing Dynamically Updating Controllers from Changes in Scenario-Based Specifications. In *Proceedings of the 7th International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS '12)*. IEEE, Piscataway, NJ, USA, 145–154. doi:10.1109/SEAMS.2012.6224401
- [19] Cristiano Giuffrida, Anton Kuijsten, and Andrew S. Tanenbaum. 2013. Safe and Automatic Live Update for Operating Systems. In *Proceedings of the Eighteenth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '13)*. ACM, New York, NY, USA, 279–292. doi:10.1145/2451116.2451147
- [20] Christopher M. Hayden, Stephen Magill, Michael Hicks, Nate Foster, and Jeffrey S. Foster. 2012. Specifying and Verifying the Correctness of Dynamic Software Updates. In *Verified Software: Theories, Tools, Experiments (Lecture Notes in Computer Science, Vol. 7152)*. Springer, Berlin, Heidelberg, 278–293. doi:10.1007/978-3-642-27705-4_22
- [21] Maurice P. Herlihy and Jeannette M. Wing. 1990. Linearizability: A Correctness Condition for Concurrent Objects. *ACM Transactions on Programming Languages and Systems* 12, 3 (1990), 463–492. doi:10.1145/78969.78972
- [22] Takanori Hirano, Takuto Yamauchi, Kenji Tei, and Shinichi Honiden. 2024. Dynamic Update Controller Synthesis using on-the-fly Exploration. *IEICE Technical Report* 123, 443 (March 2024), 76–78. In Japanese; non-peer-reviewed technical report. <https://ken.ieice.org/ken/paper/20240314Vccd/>
- [23] Martin Hofmann, Christian Neukirchen, and Harald Rueß. 2016. Certification for μ -Calculus with Winning Strategies. In *Model Checking Software (Lecture Notes in Computer Science, Vol. 9641)*. Springer, Cham, Switzerland, 111–128. doi:10.1007/978-3-319-32582-8_8
- [24] Narges Khakpour, Farhad Arbab, and Éric Rutten. 2016. Synthesizing Structural and Behavioral Control for Reconfigurations in Component-Based Systems. *Formal Aspects of Computing* 28, 1 (2016), 21–43. doi:10.1007/s00165-015-0346-y
- [25] Jeff Kramer and Jeff Magee. 1990. The Evolving Philosophers Problem: Dynamic Change Management. *IEEE Transactions on Software Engineering* 16, 11 (1990), 1293–1306. doi:10.1109/32.60317
- [26] Morten Haahr Kristensen, Thomas Wright, Cláudio Gomes, Lukas Esterle, and Peter Gorm Larsen. 2026. DynSRV: Dynamically Updated Properties for Stream Runtime Verification. In *Proceedings of the 25th International Conference on Runtime Verification (Lecture Notes in Computer Science, Vol. 16087)*. Springer, Cham, Switzerland, 101–119. doi:10.1007/978-3-032-05435-7_7
- [27] Orna Kupferman and Moshe Y. Vardi. 2001. Model Checking of Safety Properties. *Formal Methods in System Design* 19, 3 (2001), 291–314. doi:10.1023/A:1011254632723
- [28] Claus Lewerentz and Thomas Lindner. 1995. Case Study “Production Cell”: A Comparative Study in Formal Specification and Verification. In *KORSO: Methods, Languages, and Tools for the Construction of Correct Software*, Manfred Broy and Stefan Jähnichen (Eds.). Lecture Notes in Computer Science, Vol. 1009. Springer, Berlin, Germany, 388–416. doi:10.1007/BFb0015473
- [29] Jeff Magee and Jeff Kramer. 2006. *Concurrency: State Models and Java Programs* (2 ed.). John Wiley & Sons, Chichester, UK. <https://uat.store.wiley.com/en-us/concurrency-state-models-and-java-programs-2nd-edition-p-9780470093559>
- [30] Valerio Panzica La Manna, Joel Greenyer, Carlo Ghezzi, and Christian Brenner. 2013. Formalizing Correctness Criteria of Dynamic Updates Derived from Specification Changes. In *Proceedings of the 8th International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS '13)*. IEEE, Piscataway, NJ, USA, 63–72. doi:10.1109/SEAMS.2013.6595493
- [31] Jedidiah McClurg, Hossein Hojjat, Pavol Černý, and Nate Foster. 2015. Efficient Synthesis of Network Updates. In *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '15)*. ACM, New York, NY, USA, 196–207. doi:10.1145/2737924.2737980
- [32] Ross M. McConnell, Kurt Mehlhorn, Stefan Näher, and Pascal Schweitzer. 2011. Certifying Algorithms. *Computer Science Review* 5, 2 (2011), 119–161. doi:10.1016/j.cosrev.2010.09.009
- [33] Leandro Nahabedian, Víctor Braberman, Nicolás D'Ippolito, Shinichi Honiden, Jeff Kramer, Kenji Tei, and Sebastián Uchitel. 2016. Assured and Correct Dynamic Update of Controllers. In *Proceedings of the 11th International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS '16)*. ACM, New York, NY, USA, 96–107.

- doi:10.1145/2897053.2897056
- [34] Leandro Nahabedian, Víctor Braberman, Nicolás D'Ippolito, Shinichi Honiden, Jeff Kramer, Kenji Tei, and Sebastián Uchitel. 2020. Dynamic update of discrete event controllers. *IEEE Transactions on Software Engineering* 46, 11 (2020), 1220–1240. doi:10.1109/TSE.2018.2876843
- [35] Leandro Nahabedian, Víctor Braberman, Nicolás D'Ippolito, Jeff Kramer, and Sebastián Uchitel. 2022. Assured Automatic Dynamic Reconfiguration of Business Processes. *Information Systems* 104 (2022), 101850. doi:10.1016/j.is.2021.101850
- [36] Anas Nooruldeen and Klaus Werner Schmidt. 2015. State Attraction Under Language Specification for the Reconfiguration of Discrete Event Systems. *IEEE Trans. Automat. Control* 60, 6 (2015), 1630–1634. doi:10.1109/TAC.2014.2358811
- [37] Yiming Qiu, Ryan Beckett, and Ang Chen. 2023. Synthesizing Runtime Programmable Switch Updates. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI '23)*. USENIX Association, Boston, MA, USA, 613–628. <https://www.usenix.org/conference/nsdi23/presentation/qiu>
- [38] Simon Robillard and Hélène Coullon. 2022. SMT-Based Planning Synthesis for Distributed System Reconfigurations. In *Fundamental Approaches to Software Engineering (Lecture Notes in Computer Science, Vol. 13241)*. Springer, Cham, Switzerland, 268–287. doi:10.1007/978-3-030-99429-7_15
- [39] Karla Saur, Joseph Collard, Nate Foster, Arjun Guha, Laurent Vanbever, and Michael Hicks. 2016. Safe and Flexible Controller Upgrades for SDNs. In *Proceedings of the Symposium on SDN Research (SOSR '16)*. ACM, New York, NY, USA, Article 8, 12 pages. doi:10.1145/2890955.2890966
- [40] Daniel Sokolowski, Pascal Weisenburger, and Guido Salvaneschi. 2022. Change Is the Only Constant: Dynamic Updates for Workflows. In *Proceedings of the 44th International Conference on Software Engineering (ICSE '22)*. ACM, New York, NY, USA, 350–362. doi:10.1145/3510003.3510065
- [41] Sander B. Thuijsman and Michel A. Reniers. 2024. Supervisory Control for Dynamic Feature Configuration in Product Lines. *ACM Transactions on Embedded Computing Systems* 23, 5, Article 71 (2024), 25 pages. doi:10.1145/3579644
- [42] Stavros Tripakis and Karine Altisen. 1999. On-the-Fly Controller Synthesis for Discrete and Dense-Time Systems. In *Formal Methods (Lecture Notes in Computer Science, Vol. 1708)*. Springer, Berlin, Germany, 233–252. doi:10.1007/3-540-48119-2_15
- [43] Zelin Zhao, Yanyan Jiang, Chang Xu, Tianxiao Gu, and Xiaoxing Ma. 2021. Synthesizing Object State Transformers for Dynamic Software Updates. In *Proceedings of the 43rd IEEE/ACM International Conference on Software Engineering (ICSE '21)*. IEEE, Los Alamitos, CA, USA, 1111–1122. doi:10.1109/ICSE43902.2021.00103
- [44] Sebastián A. Zudaire, Leandro Nahabedian, and Sebastián Uchitel. 2022. Assured Mission Adaptation of UAVs. *ACM Transactions on Autonomous and Adaptive Systems* 16, 3–4, Article 7 (2022), 27 pages. doi:10.1145/3513091